databricks
Academy

Lecture - APPENDIX

Duration: 20 minutes

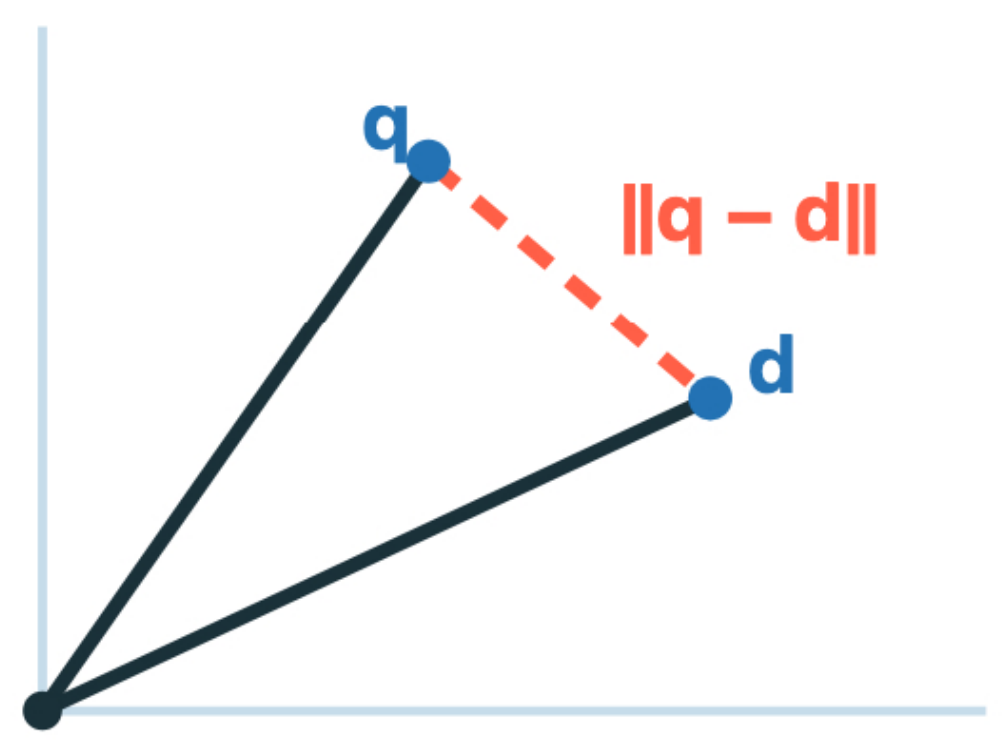
This appendix covers the core search primitives that power retrieval in a RAG system on Databricks. Each section pairs a visual reference diagram with the underlying concepts:

- **Distance Metrics** — how Lakebase measures vector similarity (L2, cosine, inner product) and which operator class to choose.
- **Approximate Nearest-Neighbor (ANN) Search** — how `lakebase_ann` indexes partition the vector space to trade a small amount of accuracy for large speedups over exact KNN.
- **Lakebase ANN Parameters** — the session-level knobs (`probes` , `epsilon` , `prefilter`) that tune recall vs. speed at query time.
- **BM25 Full-Text Search** — how `lakebase_bm25` indexes rank documents by lexical relevance using term frequency, inverse document frequency, and length normalization.

Distance Metrics

When you create a `lakebase_ann` index in Lakebase, you choose an **operator class** that determines how similarity between two vectors is measured. The choice should match how your embedding model was trained — most text embedding models use cosine similarity, but Euclidean (L2) and inner product are available when your use case requires them.

See: AWS | Azure



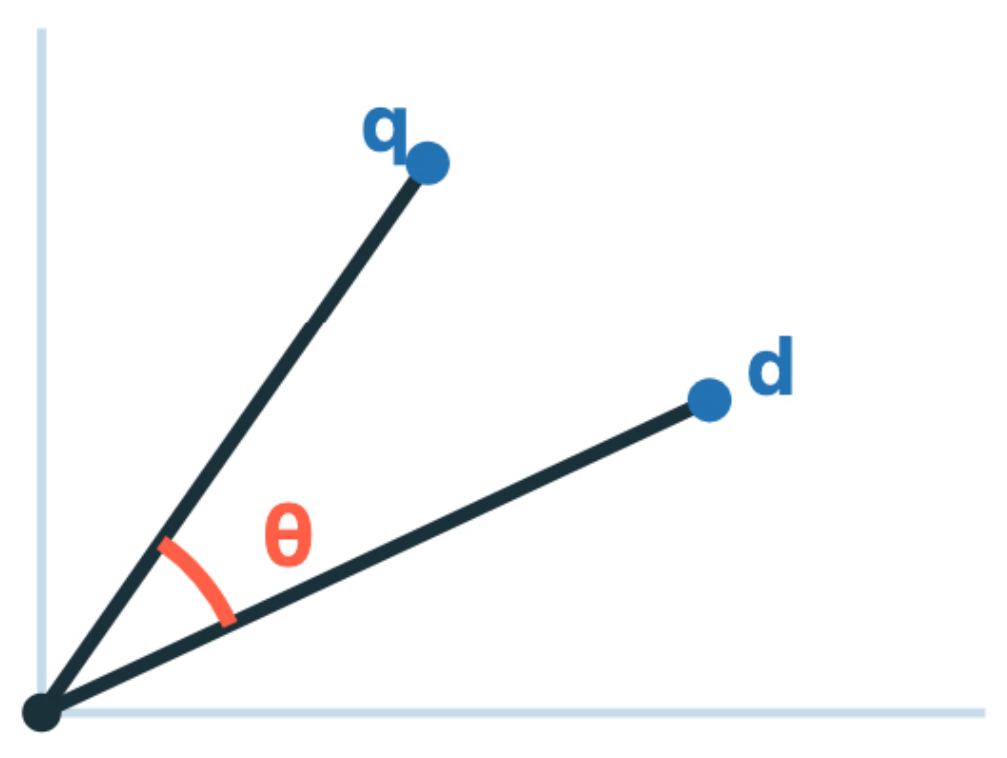
L2 · Euclidean

<->

`vector_l2_ops`

Straight-line distance between the two points. Use when **absolute spatial distance** matters and vectors are **not normalized**.

MOST COMMON

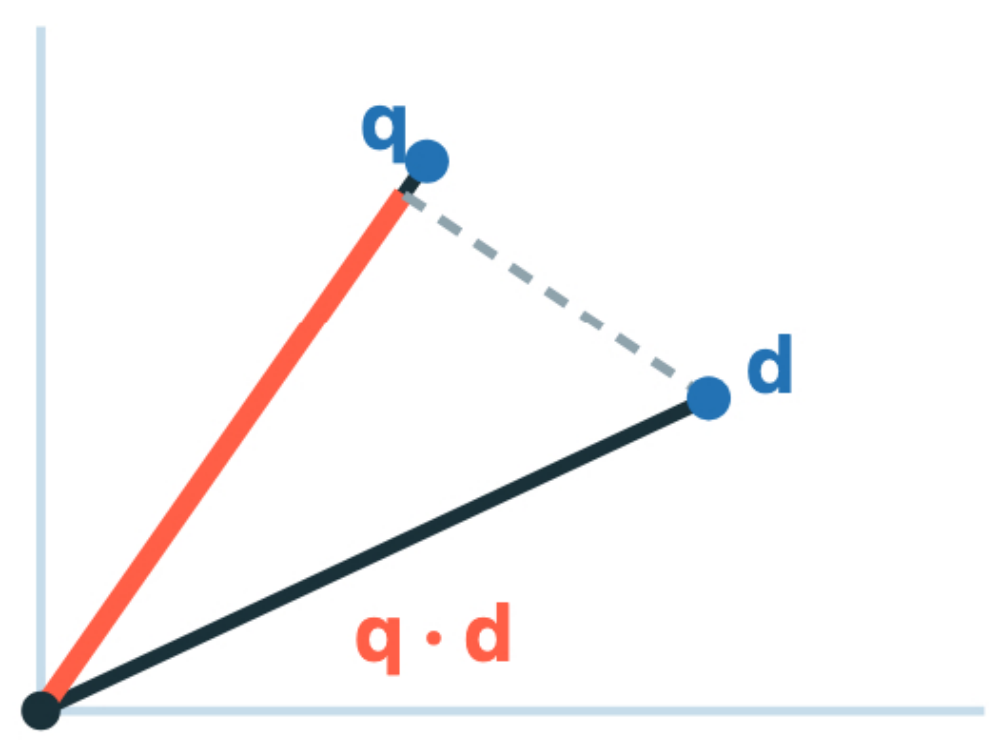


Cosine similarity

<=>

`vector_cosine_ops`

The **angle** between vectors, ignoring magnitude. Use for **most text embeddings** — the default choice.



Inner product

<#>

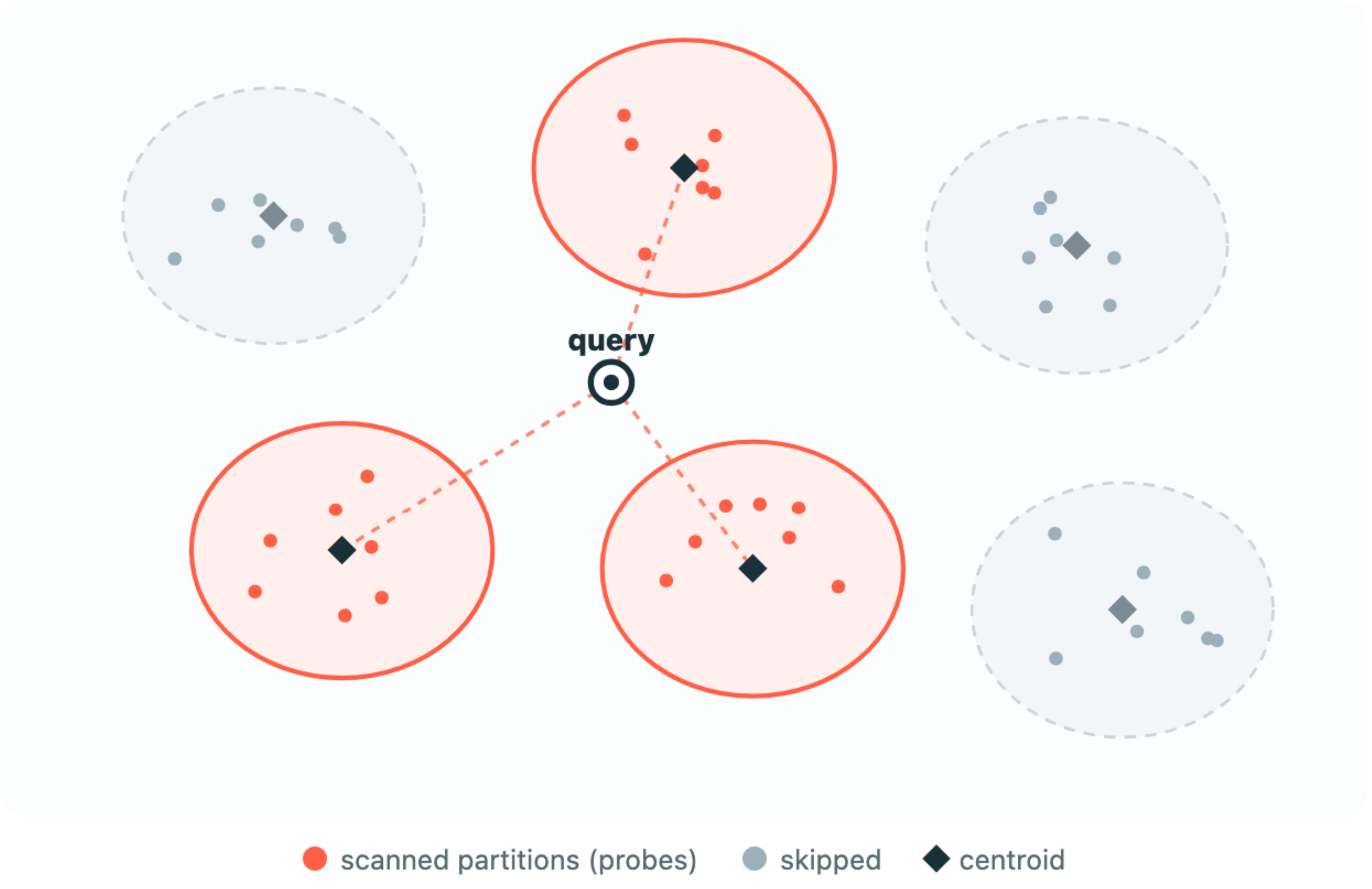
`vector_ip_ops`

Projection of one vector onto the other. Use when vectors are **pre-normalized** to unit length — faster than cosine, same ranking.

Approximate Nearest-Neighbor (ANN) Vector Search

Exact nearest-neighbor search compares the query vector against every row in the table — accurate, but far too slow for production workloads. Lakebase's `lakebase_ann` index uses an **Inverted File (IVF)** structure that clusters vectors into partitions around centroids at build time. At query time, only the partitions whose centroids are closest to the query are scanned. This trades a small amount of recall for dramatically faster search, making it practical to search millions of vectors in milliseconds.

See: AWS | Azure



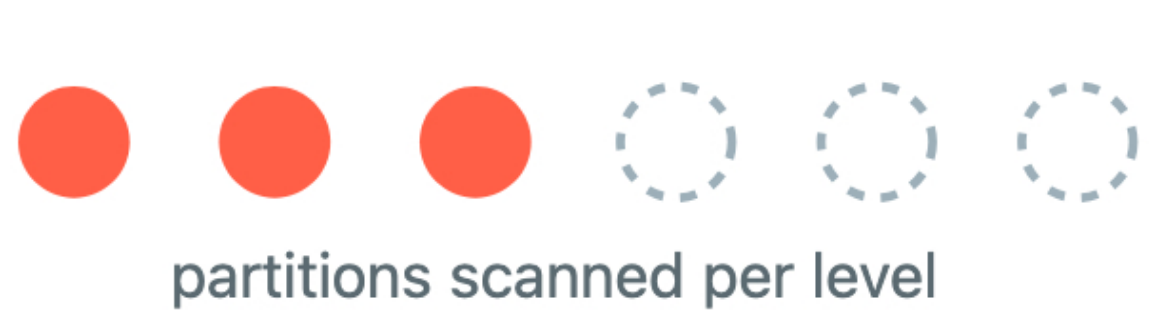
HOW IT WORKS

- **Partition once.** At build time, vectors are clustered into lists around centroids (IVF).
- **Scan a few.** A query is compared to centroids, then only the nearest partitions are searched — not the whole table.
- **Recall ↔ speed.** Scanning more partitions raises recall but costs query time. ANN trades a little accuracy for large speedups over exact KNN.

Lakebase ANN Parameters

After creating a `lakebase_ann` index, you can tune the **recall vs. speed** trade-off at query time using three session-level settings (GUCs). The defaults (`'auto'`) work well in most cases, but you can override them per session when you need higher recall for evaluation benchmarks or lower latency for production serving.

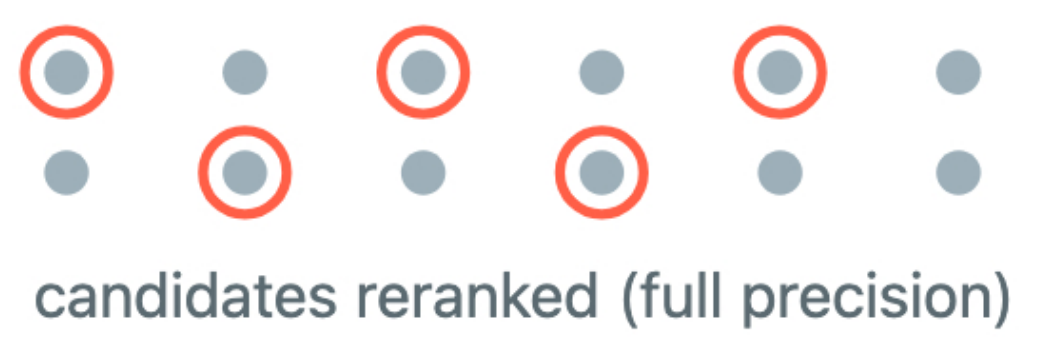
See: AWS | Azure



`lakebase_ann.probes` default `'auto'`

Number of IVF partitions to scan at each level. Higher values improve recall at the cost of query speed.

↑ recall ↓ speed



`lakebase_ann.epsilon` default `'auto'`

Controls how many candidates are reranked using full-precision distances. Higher values rerank more candidates and take longer.

↑ recall ↓ speed

`lakebase_ann.prefilter` default `off`

Evaluates non-vector filters *before* full-precision reranking. Valid values `on` / `off` .

best: cheap filter · removes most rows

BM25 Full-Text Search

Vector search finds semantically similar content, but sometimes you need **exact keyword matching** — for product names, error codes, proper nouns, or any term where spelling matters more than meaning. Lakebase's `lakebase_bm25` index provides production-grade full-text search that ranks results using the BM25 algorithm, which improves on Postgres's built-in `ts_rank` by incorporating corpus-wide statistics for more stable, accurate scoring as data grows.

See: AWS | Azure

BM25 SCORE = 3 FACTORS

Term frequency
How often the term appears in a document — more is more relevant.

Inverse document frequency
Rare terms across the corpus weigh more than common ones.

Length normalization
Long documents don't win just for being long.

More accurate than `ts_rank` , which lacks global corpus statistics, so scores stay stable as data grows.

INDEX & QUERY

①
```
to_tsvector('english', passage)
```
Tokenize text — stemming & stop-word removal for the language.

②
```
CREATE INDEX ... USING lakebase_bm25 (vector)
```
Build **after** inserting data — BM25 computes corpus stats at build time.

③
```
vector <@> to_bm25query(...)
```
Returns a **negative** score — `ORDER BY score` ascending for best matches first.

④
```
Block-Max WAND → top-K
```
Returns only the K most relevant rows without scoring every match.

RANKED RESULTS `query: vector database`

① building a scalable vector database for semantic search 9.4

② the database stores embeddings and metadata 5.8

③ a general-purpose data store for applications 2.6

© 2026 Databricks, Inc. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.